# 7 Konstruktoren und Destruktoren

## 7.1 Konstruktoren

Der "**Konstruktor**" einer Klasse ist eine besondere Elementfunktion. Sie wird bei der Erzeugung (Definition) eines jeden Objekts dieser Klasse automatisch aufgerufen. Üblicherweise führt der Konstruktor die Initialisierung des Objekts durch.

- Eine Elementfunktion ist genau dann ein Konstruktor, wenn sie denselben Namen wie die Klasse besitzt.
- Ein Konstruktor wird wie eine gewöhnliche Elementfunktion programmiert, es fehlt allerdings der Return-Typ.

- In C können Strukturvariable durch eine Werteliste (in geschweiften Klammern) initialisiert werden. Point p = {1 5, 7}
  Dies ist für struct- und class-Typen in C++ nicht mehr möglich, wenn die Klasse
  – geschützte oder private Elemente oder einen Konstruktor besitzt oder
  – abgeleitet ist oder virtuelle Elementfunktionen (s.u.) besitzt.

- Wenn eine Klasse keinen eigenen Konstruktor definiert (und nur dann), fügt der Compiler automatisch einen parameterlosen Konstruktor ohne besondere Funktionalität hinzu.
- Ein Konstruktor, der sich parameterlos <u>aufrufen</u> läßt, heißt "Default-Konstruktor", weil er in einigen Fällen standardmäßig (automatisch) aufgerufen wird.

- Normalerweise sind Konstruktoren public.
- Konstruktoren werden nicht vererbt.

- Konstruktoren sind nicht für die Speicherallokation verantwortlich. Der Speicher für ein neues Objekt ist bereits reserviert bevor der Konstruktor ausgeführt wird.

Der Konstruktor einer Klasse wird insbesondere in folgenden Fällen aufgerufen:

1. Bei jeder Definition eines Objekts der Klasse

   Syntax: `Klassenname objekt (Parameterliste);`

   ohne Parameter zwingend: `Klassenname objekt;`

2. Beim Erzeugen eines Objekts mit new

   Syntax: `new Klassenname (Parameterliste)`

   ohne Parameter alternativ: `new Klassenname`

3. In Ausdrücken bei explizitem Aufruf des Konstruktors (wie eine C-Funktion)

   Syntax: `Klassenname (Parameterliste)`

   Der "Wert" eines solchen Konstruktoraufrufs ist das von ihm erzeugte Objekt, das aber nur temporär während der Auswertung des Ausdrucks existiert.

   Sonderfall:
   Da in C (und C++) Variablen auch durch einen (typkompatiblen) Ausdruck initialisiert werden können, ist somit alternativ zu Fall 1 auch die Syntax

   `Klassenname objekt = Klassenname (Parameterliste)`

   möglich.

4. Bei der Konstruktion einer abgeleiteten Klasse

   Syntax für Konstruktor<u>definition</u> der abgeleiteten Klasse:

```
AbgeleiteteKlasse (Parameterliste)
                        : Basisklasse (aktuelle Parameter)   optionell
{
  Konstruktorcode für abgeleitete Klasse
  (oft nur Initialisierung der zusätzlichen Elemente)
}
```

   Bei Auslassung des Basisklassenkonstruktor-Aufrufs wird trotzdem <u>immer</u> ein Konstruktor der Basisklasse aufgerufen, und zwar der Defaultkonstruktor.

5. Bei der Konstruktion einer Klasse, die ein Objekt als Elementvariable enthält.
   Syntax für Konstruktordefinition einer solchen Klasse:

```
UmschließendeKlasse (Parameterliste)
                        : Elementname (aktuelle Parameter)   optionell
{
  Konstruktorcode für umschließende Klasse
}
```

   Bei Auslassung des obigen Elementkonstruktor-Aufrufs wird für das Element trotzdem immer ein Konstruktor aufgerufen, und zwar der Defaultkonstruktor.

   Hinweis: In Anlehnung an diese Syntax lassen sich auch Elemente initialisieren, die nicht Objekte, sondern einfache Typen sind. Statt dem "aktuellen Parameter" ist dann der Initialwert anzugeben.

6. Ein Array von Objekten kann man bei der Definiton wie in C mit einer Initialisierungsliste (Werteliste in geschweiften Klammern) vorbelegen. Für ausgelassene Elemente wird der Defaultkonstruktor aufgerufen. Ohne Initialisierungsliste wird für jedes Arrayelement der Defaultkonstruktor aufgerufen.

- Nach der Parameterliste eines Konstruktors und dem Doppelpunkt können in komplizierten Fällen sowohl der Basisklassenkonstruktor als auch mehrere Elementkonstruktoren in einer kommagetrennten Liste aufgeführt werden.

- Die Konstruktionsreihenfolge ist immer von innen nach außen (evtl. rekursiv):
  1. Schritt: Konstruktion des Basisklassenobjekts
  2. Schritt: Konstruktion der Element-Objekte
  3. Schritt: Ausführung des eigenen Konstruktorcodes.

- Globale Variablen werden vor Aufruf von `main` konstruiert.

## Konstruktor

**Beispiel zu 7.1**

```
#include <iostream>
using namespace std;

class Artikel {
  public:
    Artikel(int _bestellnr, int _anzahl) {    // 1. Konstruktor
      cout << "Signatur int, int\n";                          // Test
      bestellnr = _bestellnr; anzahl = _anzahl; }
    Artikel(int _bestellnr) {                 // Typumwandlungskonstr.
      cout << "Signatur int\n";                               // Test
      bestellnr = _bestellnr; anzahl = 1; }
    Artikel(const Artikel& a) {               // Kopierkonstruktor
      cout << "Kopierkonstruktor\n";                          // Test
      bestellnr = a.bestellnr+1; anzahl = a.anzahl; }
    void print() {                            // Ausgabefunktion
      cout << "Nr " << bestellnr
           << ": Vorrat " << anzahl << endl; }
    int bestellen(int menge);
  protected:
    int bestellnr;
    int anzahl;
};

int Artikel::bestellen(int menge) {
  if (anzahl < menge) menge = anzahl;
  anzahl = anzahl - menge;
  return menge; }

// Hauptprogramm
int main() {
 Artikel* p;
 Artikel cdrom(1000, 4);              //
   cdrom.print();
 p = new Artikel(1001, 7);            //
   p->print();
 cdrom = Artikel(1003);               //
   cdrom.print();
 Artikel monitor = Artikel(1005,5);   //
   monitor.print();
 monitor = Artikel(cdrom);            //
   monitor.print();
}
```

Ausgabe:

1 [ Signatur int, int
Nr. 1000 , Vorrat 4

2 [ Signatur int, int
Nr. 1001 : Vorrat 7

5 [ Signatur int
Nr. 1003 : Vorrat 1

4 [ Signatur int, int
Nr 1005 : Vorrat 5

5 [ kopierkonstruktor
Nr. 1004 : Vorrat 1

## 7.2 Destruktoren

Der "**Destruktor**" einer Klasse ist eine besondere Elementfunktion, die automatisch aufgerufen wird, wenn die Lebensdauer eines Objekts endet.

- Eine Elementfunktion ist genau dann der Destruktor, wenn sie den Namen der Klasse - mit vorangestellter Tilde - trägt.
- Der Destruktor wird wie eine gewöhnliche Elementfunktion programmiert, es fehlt allerdings der Return-Typ. Auch darf er keine Parameter besitzen.
  Prototyp:

```
~Klassenname();
```

- Wenn eine Klasse keinen eigenen Destruktor definiert, fügt der Compiler automatisch einen "Default-Destruktor" ohne besondere Funktionalität hinzu.

- Normalerweise sind Destruktoren public.
- Destruktoren werden nicht vererbt.

- Destruktoren sind nicht für die Speicherfreigabe verantwortlich. Diese erfolgt automatisch nachdem der Destruktorcode ausgeführt worden ist.

# 7.3 Kopierkonstruktor und Typumwandlungskonstruktor

Wenn der Konstruktor einer Klasse mit einem Parameter aufgerufen wird lassen sich zwei Fälle unterschieden:

1.) Der Parameter besitzt den Typ der Klasse

2.) Der Parameter besitzt einen anderen Typ als die Klasse

Ein solcher Konstruktor wird in folgenden Fällen implizit aufgerufen:

a) bei Initialisierung eines Objekts der Klasse mit einem Ausdruck

b) bei Zuweisung an ein Objekt der Klasse (nur sofern Typwandlung nötig)

c) bei Parameterübergabe an eine Funktion, wenn der formale Parameter ein Objekt der Klasse ist (nicht bei Referenzparametern)

d) zur Bildung des Returnwertes einer Funktion, wenn der Returntyp die Klasse ist (nicht bei Referenz)

Für Kopierkonstruktoren gilt zusätzlich:

- Wenn eine Klasse keinen eigenen Kopierkonstruktor definiert, fügt der Compiler automatisch einen Kopierkonstruktor hinzu, der eine elementweise Kopie aller Elementvariablen erzeugt.
- Jeder Kopierkonstruktor muss einen Referenzparameter besitzen!
- Um in manchen Fällen schwer zu diagnostizierende Fehler zu vermeiden, sollte der Parameter außerdem mit 'const' deklariert werden.

## Klasse rationaler Zahlen: Konstruktoren und friends

**Beispiel zu 7.3**

```
#include <iostream>
using namespace std;

class Ratio {
    int zaehler;
    int nenner;
  public:
    // Konstruktoren
    Ratio (int _zaehler = 0, int _nenner = 1) {
      zaehler = _zaehler; nenner = _nenner;
      cout << "Konstruktor fuer "; print(); }
    Ratio (const Ratio &x) {
      zaehler = x.zaehler; nenner = x.nenner;
      cout << "Kopierkonstr. fuer "; print(); }
    // Ausgabefunktion
    void print() { cout << zaehler << '/' << nenner << endl;}
    // Multiplikation mit Elementfunktion
    Ratio mult (Ratio y) {
      return Ratio (zaehler*y.zaehler, nenner*y.nenner); }
    friend Ratio mult (Ratio, Ratio);       // zwingend wegen ::mult()
};

// Multiplikation mit C-Funktion
Ratio mult (Ratio x, Ratio y) {
  return Ratio (x.zaehler*y.zaehler, x.nenner*y.nenner); }

int main() {
  Ratio b;                                  //
  b = Ratio (5, 2);                         //
  Ratio a = 9;                              //
  Ratio c = b;                              //
  a = 3;                                    //
//----- Mult. mit Elementfunktion
  c = b.mult(a);                            //
                                            //
  c.print();                    //
  c = b.mult(Ratio(3,4));                   //
                                            //
  c.print();                    //
//----- Mult. mit C-Funktion
  c = mult(b, a);                           //
                                            //
                                            //
  c.print();                    //
  c = mult(b, Ratio(3,4));                  //
                                            //
                                            //
  c.print();                    //
}
```

# 8 Virtuelle Funktionen

## 8.1 Prinzip

Nach den bisher behandelten Regeln (Kap. 5.2) entscheidet allein die Deklaration einer Variablen über die Klasse, aus der eine Elementfunktion aufgerufen wird.

Wenn es sich um einen Zeiger handelt, so kann dieser zur Laufzeit bekanntlich auch auf Instanzen abgeleiteter Klassen zeigen. In diesem Fall möchte man oft, dass der zur Laufzeit referenzierte Typ (und nicht der Deklarationstyp des Zeigers) darüber entscheidet, welche Elementfunktion aufgerufen wird.

Genau dies geschieht, wenn die Elementfunktion "**virtuell**" ist.

- Eine Elementfunktion wird dadurch virtuell, dass man ihrer Deklaration das Schlüsselwort "virtual" voranstellt. Damit sind auch alle gleichnamigen Funktionen in abgeleiteten Klassen virtuell, sofern sie in Signatur und Returntyp übereinstimmen, d.h. wenn sie die Funktion aus der Basisklasse überschreiben.
- Das Schlüsselwort "virtual" ist in den abgeleiteten Klassen optionell.

- Genau dann, wenn die nach den Gültigkeitsbereichsregeln ausgewählte Elementfunktion virtuell ist <u>und</u> entweder
  - der Aufruf über einen Zeiger (oder eine Referenz) erfolgt oder
  - der Aufruf ohne Objektangabe erfolgt (nur innerhalb Elementfunktion möglich!),

  bestimmt der Laufzeittyp des aktuellen Objekts[1] die Klasse der tatsächlich aufgerufenen Elementfunktion.

- Ist eine Funktion in abgeleiteten Klassen überschrieben, ist es normalerweise sinnvoll, sie als virtuell zu deklarieren.
- Eine Klasse mit virtuellen Funktionen wird gelegentlich "**polymorphe Klasse**" genannt.

- Konstruktoren können nicht als virtuell deklariert werden, wohl aber Destruktoren.

---

[1] Bei Aufruf einer virtuellen Funktion über Zeiger/Referenz ist dies der Typ, auf das der Zeiger oder die Referenz verweist.

Bei Aufruf einer virtuellen Funktion ohne Objektangabe ist dies der Typ des aktuellen Objekts der aufrufenden Methode (d.h. der Typ, auf den der 'this'-Zeiger verweist, s.u.).

## Weshalb virtuelle Funktionen?

```
#include <iostream>
using namespace std;

class Artikel {
  public:
    Artikel(int _bestellnr, int _anzahl) {
      bestellnr = _bestellnr; anzahl = _anzahl; }
    void print() {
      cout << "Nr " << bestellnr
           << ": Vorrat " << anzahl << endl; }
    int bestellen(int menge);
  protected:
    int bestellnr;
    int anzahl;
};
int Artikel::bestellen(int menge) {
  if (anzahl < menge) menge = anzahl;
  anzahl = anzahl - menge;
  cout << "Aenderung ";                                    // neu
  print();
  return menge; }

class Hdisk: public Artikel {
    int kap;
  public:
    Hdisk (int _best, int _anz, int _kap) : Artikel (_best, _anz) {
      kap = _kap;
    }
    void print() {
      cout << "Nr " << bestellnr << ": Vorrat "
           << anzahl << ", " << kap << " GB" << endl;
    }
};

// Hauptprogramm
int main() {
  Artikel cdrom (2133, 4);
  Hdisk western (1122, 8, 400);
  Artikel *p;
  p = &cdrom;
  p->print();          //
  p = &western;
  p->print();          //
  cdrom.bestellen(3);  //
  western.bestellen(2); //
}
```

## Verwendung einer virtuellen Funktion

**Beispiel zu 8.1**

```
#include <iostream>
using namespace std;

class Artikel {
  public:
    Artikel(int _bestellnr, int _anzahl) {
      bestellnr = _bestellnr; anzahl = _anzahl; }
    virtual void print() {                          // jetzt virtuell
      cout << "Nr " << bestellnr
           << ": Vorrat " << anzahl << endl; }
    int bestellen(int menge);
  protected:
    int bestellnr;
    int anzahl;
};
int Artikel::bestellen(int menge) {
  if (anzahl < menge) menge = anzahl;
  anzahl = anzahl - menge;
  cout << "Aenderung ";
  print();
  return menge; }

class Hdisk: public Artikel {
    int kap;
  public:
    Hdisk (int _best, int _anz, int _kap) : Artikel (_best, _anz) {
      kap = _kap;
    }
    void print() {
      cout << "Nr " << bestellnr << ": Vorrat "
           << anzahl << ", " << kap << " GB" << endl;
    }
};

// Hauptprogramm
int main() {
  Artikel cdrom (2133, 4);
  Hdisk western (1122, 8, 400);
  Artikel *p;
  p = &cdrom;
  p->print();          //
  p = &western;
  p->print();          //
  cdrom.bestellen(3);  //
  western.bestellen(2); //
}
```

## Ergänzung: Virtueller Destruktor

*X* sei eine Klasse, ein Zeiger *p* sei vom Typ "*X**".
Der Zeiger kann dann bekanntlich auf eine dynamisch erzeugte Instanz einer von *X* abgeleiteten Klasse verweisen.

Damit in einem solchen Fall der dynamische Speicher mit "`delete p;`" korrekt freigegeben wird, muss gemäß C++ Sprachstandard die Klasse *X* einen "virtuellen Destruktor" besitzen.
Es genügt dazu bereits, wenn der Destruktor einen leeren Rumpf hat, also

```
virtual ~X() {}
```

Die von *X* abgeleitete Klasse muss dabei nicht unbedingt einen eigenen Destruktor definieren.

# 8.2 Abstrakte Basisklassen und rein virtuelle Funktionen

Manchmal hat man es mit einer Hilfsklasse zu tun, von der keine Instanzen erzeugt werden sollen. Man kann dies vom Compiler überwachen lassen. Dazu muss die Klasse eine **"abstrakte Basisklasse"** sein.

- Eine Klasse ist genau dann abstrakt, wenn sie mindestens eine **"rein virtuelle Funktion"** ("pure virtual function") besitzt.
- Eine virtuelle Funktion wird dadurch zu einer rein virtuellen Funktion, dass ihr Funktionsrumpf durch die Notation "=0" ersetzt wird. Sie besitzt also keine konkrete Implementierung.
- Von einer abstrakten Klasse lassen sich keine Objekte erzeugen.
- Jede von einer abstrakten Klasse abgeleitete Klasse ist gezwungen, für alle ererbten rein virtuellen Funktionen Implementierungen zu definieren. Andernfalls ist sie selbst wieder abstrakt.

# 9 Weitere Eigenschaften von C++

## 9.1 Der Zeiger 'this'

In jeder[1] Elementfunktion existiert eine vordefinierte Variable '**this**'.
Sie ist ein Zeiger auf das aktuelle Objekt.
Mit '`*this`' kann also das aktuelle Objekt selbst angesprochen werden.

Der Datentyp von 'this' in einer Klasse *X* ist

```
X* const
```

Wenn ein Elementname *elem* in einer Elementfunktion alleine steht, ist dies exakt identisch zu den Ausdrücken

| | |
|---|---|
| `this->elem` | falls *elem* Elementvariable bzw. |
| `this->elem(..)` | falls *elem* Elementfunktion. |

[1] genauer: In jeder nichtstatischen Elementfunktion

# Hintergrundinfo zur Realisierung von Methodenaufrufen

1. Eine Elementfunktion wird wie jede C-Funktion genau einmal in den Hauptspeicher geladen. Alle Instanzen der Klasse benutzen daher denselben Maschinencode.
   Damit eine Methode weiß, auf welche Elementvariablen sie zugreifen muss, wird bei jedem Methodenaufruf der Zeiger auf das aktuelle Objekt als (unsichtbarer) zusätzlicher Parameter übergeben. Dieser Zeiger steht innerhalb der Elementfunktion auch dem Programmierer als this-Zeiger zur Verfügung.
   Jede Verwendung eines Elements `elem` ist daher äquivalent zu
   `this->elem` (für Elementvariablen) bzw.
   `this->elem(....)` (für Elementfunktionen)
   und wird auch genau in dieser Form vom Compiler übersetzt.

Beispiel:

C++ - Code

```
class Artikel {

  ....

  int bestellen (int menge) {
    anzahl = anzahl - menge;
  }
};

int main() {
  Artikel neonlight;
  ....
  neonlight.bestellen(3);
  ....
}
```

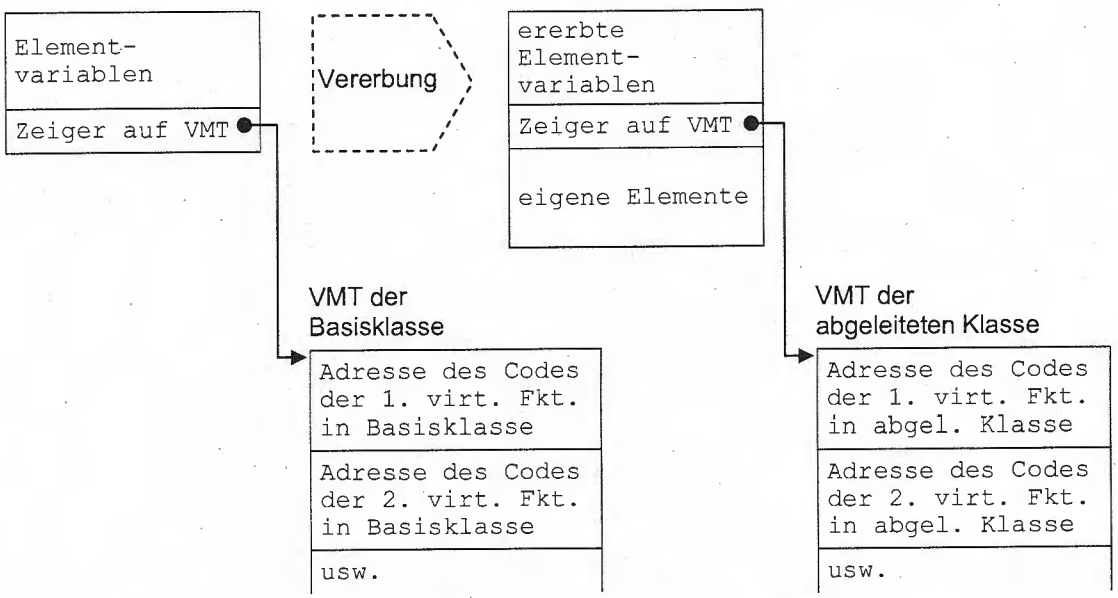
äquivalenter C-Code

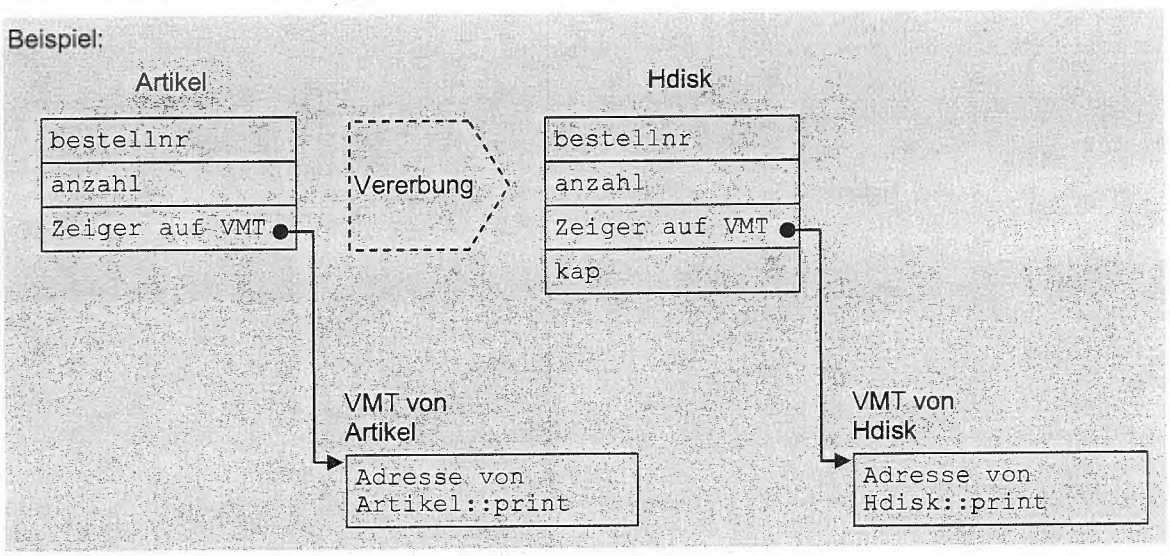
```
struct Artikel {
  int anzahl;
  ....
};
int bestellen (Artikel* this, int menge) {
  this->anzahl = this->anzahl - menge;
}


int main() {
  Artikel neonlight;
  ....
  bestellen(&neonlight, 3);
  ....
}
```

2. Zu jeder Instanz wird Speicher für alle (nichtstatischen) Elementvariablen reserviert. Ererbte Elemente kommen zuerst und haben dieselbe Anordnung wie in der Basisklasse.

Beispiel:

3. In jeder Basisklasse mit virtueller Funktion wird eine zusätzliche (unsichtbare) Elementvariable angelegt, die auf eine "virtuelle Methodentabelle" (VMT) zeigt. Der Aufruf von virtuellen Methoden erfolgt nicht direkt sondern in drei Schritten:
   - Zuerst wird im aktuellen Objekt der Zeiger auf die VMT gelesen.
   - Dann wird in der VMT die Adresse der gewünschten Funktion gelesen.
   - Schließlich wird diese Funktion aufgerufen.




4. Eine statische Elementvariable wird nicht für jede Instanz angelegt sondern vom Compiler wie eine einzige globale C-Variable behandelt. Alle Methoden der Klasse greifen daher auf denselben Wert zu.

## 9.2 Friends

Wenn Klassen eng kooperieren und oft wechselseitig die Werte von Elementvariablen benötigen, müssen normalerweise öffentliche Zugriffsfunktionen implementiert werden. Dies ist sowohl ineffizient als auch programmieraufwändig.

Für solche Fälle kann eine Klasse anderen Elementfunktionen, aber auch C-Funktionen, Zugriff auf alle ihre Elemente gewähren, indem Sie diese als 'friend' deklariert durch folgende Deklaration innerhalb der Klasse

| | |
|---|---|
| `friend` *`Prototyp`*`;` | Zugriff erhält die angegebene Funktion |
| `friend class` *`Klassenname`*`;` | Zugriff erhalten alle Methoden der Klasse |

# 9.3 Die Klasse 'string' der Standardbibliothek

#include <string> mit namespace std notwendig!

- Wichtige Elementfunktionen: (Abkürzung: Typ T = char*, string&
  Typ TT = char*, string&, char)

  - Konstruktoren:

| | | |
|---|---|---|
| `string();` | string s, s1; | // Konstruktor für Leerstring |
| `string(T);` | string s2 ("Di Mi "); | |
| | string s3 (s2); | // Kopierkonstruktor |

  - sonstige Elementfunktionen:

| | |
|---|---|
| `int size();` | Returnwert: Anzahl der Zeichen im String |
| `const char* c_str();` | Zeiger auf internen C-String beschaffen. Returnwert: Zeiger auf nullterm. C-String, Änderung des Strings macht Zeiger ungültig! |
| `int copy(char*,int n);` | Kopieren von maximal n Zeichen des Strings in ausreichend dimensioniertes (!) char-Array. Returnwert: Zahl kopierter Zeichen |
| `int find(TT s);` | Suchen des ersten Auftretens von s im String. Returnwert: Indexposition des Treffers, bei Fehlschlag string::npos |
| `string substr(int a,int anz);` | Erzeugen eines Teilstrings mit anz Zeichen ab Index a. Returnwert: Teilstring |
| `string replace (int a,int anz,T s);` | Ersetzen von anz Zeichen ab Index a durch s. Returnwert: veränderter String |

- Wichtige Operationen (Operandentyp T, mindestens ein Operand string):

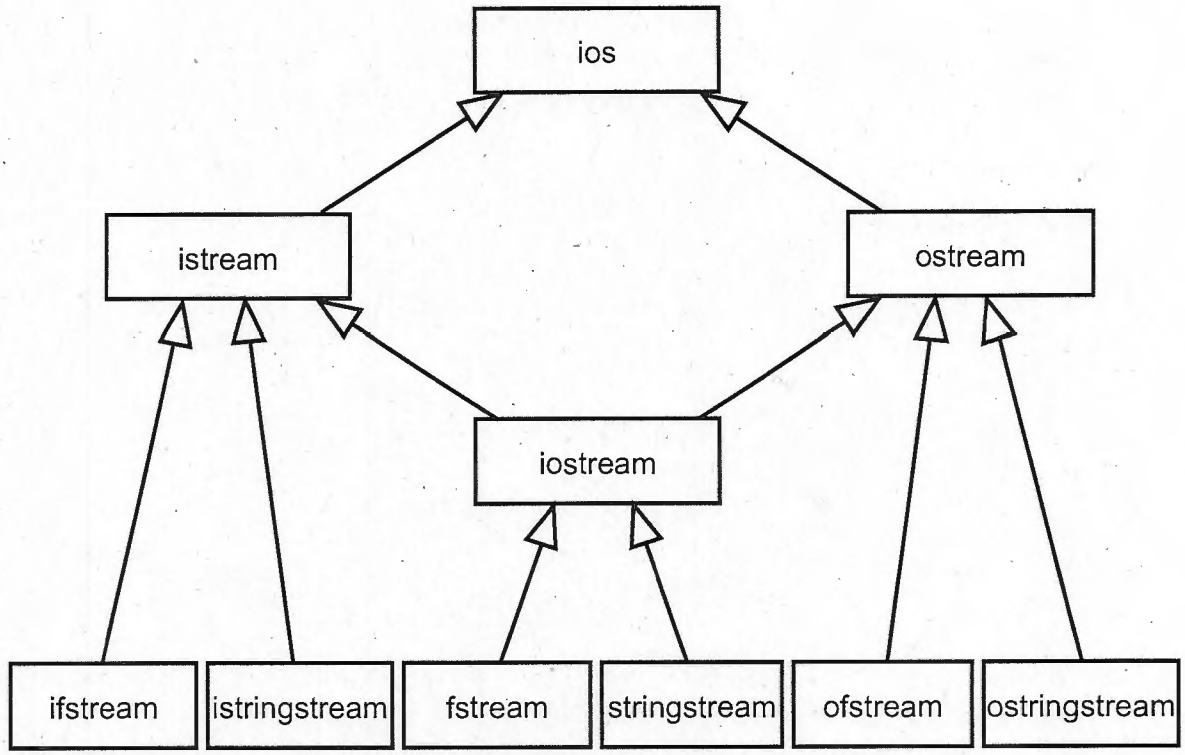
| | | | |
|---|---|---|---|
| = | Zuweisung | s1 = "So "; | |
| += | Anfügen | s1 += "Mo "; | |
| + | Anfügen | s1 = s1 + s2 + "Do "; | |
| << | Ausgabe | cout << s1; | |
| >> | Eingabe | cin >> s2; | // nur bis Whitespace |
| [int] | Zeichenzugriff | char c = s1[0]; | // Indexzählung ab 0 |
| == != | Vergleiche | if (s1 == s2) ... | |

- Wichtige C-Funktion:

| | |
|---|---|
| `istream& getline(istream&, string&, char term = '\n');` | Angegebenen istream bis Zeichen term in String einlesen (ohne Zeichen term selbst). |

# 9.4 Weitere Klassen für Ein/Ausgabe

## 9.4.1 Hierarchie der Stream-Klassen (vereinfacht)



## 9.4.2 Datei-Ein/Ausgabe

Klassen: ofstream (Ausgabe), ifstream (Eingabe), fstream (Ein- und Ausgabe)

#include <fstream> mit namespace std notwendig!

- Wichtige Elementfunktionen von ofstream
  - Konstruktoren:

| | |
|---|---|
| `ofstream();` | Erzeugen eines Filestream-Objekts (ist danach mit Elementfunktion open an eine Datei zu binden) |
| `ofstream(char* s,int modus = ios::out\|ios::trunc);` | Erzeugen eines Filestream-Objekts, verbinden mit Datei s und öffnen mit angegebenem Modus (z.B. ios::app, ios::trunc, ios::out, ios::in) |

  - sonstige Elementfunktionen:

| | |
|---|---|
| `void open(char* s,int modus = ios::out\|ios::trunc);` | Öffnen der Datei s mit dem Filestream (s.o.) |
| `void close();` | Schließen der Datei (Filestream existiert weiter) |
| `ostream& write(char* bytes, int anzahl);` | Binäres Schreiben der angegebenen Bytes |
| `~ofstream();` | Destruktor, schließt ggf. noch offene Datei |
| ferner `width`, `setf`, `fail` usw. | Ererbt von ostream |

- Wichtige Elementfunktionen von ifstream
  - Konstruktoren:

| | |
|---|---|
| `ifstream();` | Erzeugen eines Filestream-Objekts (ist danach mit Elementfunktion open an eine Datei zu binden) |
| `ifstream(char* s,int modus = ios::in);` | Erzeugen eines Filestream-Objekts, verbinden mit Datei s und öffnen mit angegebenem Modus (z.B. ios::in, ios::out) |

  - sonstige Elementfunktionen:

| | |
|---|---|
| `void open(char* s,int modus = ios::in);` | Öffnen der Datei s mit dem Filestream (s.o.) |
| `void close();` | Schließen der Datei (Filestream existiert weiter) |
| `istream& read(char* bytes, int anzahl);` | Binäres Lesen der angegebenen Bytes |
| `~ifstream();` | Destruktor, schließt ggf. noch offene Datei |
| ferner `get`, `getline`, `fail` usw. | Ererbt von istream |

- Operatoren und Manipulatoren — Ererbt von ostream bzw. istream

## 9.4.3 Formatiertes Lesen und Schreiben von C++-Strings

Klassen: ostringstream (Schreiben), istringstream (Lesen)
Anwendungsbeispiel: Konvertierung von Zahlen in C++-Strings und umgekehrt

#include <sstream> mit namespace std notwendig!

- Wichtige Elementfunktionen von ostringstream
  - Konstruktoren:

| | |
|---|---|
| `ostringstream();` | Erzeugen eines Objekts ohne Inhalt |
| `ostringstream(string& s);` | Erzeugen eines Objekts mit string s als Inhalt |

  - sonstige Elementfunktionen:

| | |
|---|---|
| `string str();` | Liefert Inhalt des Objekts als string |
| `void str(string& s);` | Initialisiert Objekt neu mit string s als Inhalt |
| ferner `width`, `setf` usw. | Ererbt von ostream |

- Wichtige Elementfunktionen von istringstream
  - Konstruktoren:

| | |
|---|---|
| `istringstream();` | Erzeugen eines Objekts ohne Inhalt |
| `istringstream(string& s);` | Erzeugen eines Objekts mit string s als Inhalt |

  - sonstige Elementfunktionen:

| | |
|---|---|
| `string str();` | Liefert Inhalt des Objekts als string |
| `void str(string& s);` | Initialisiert Objekt neu mit string s als Inhalt |
| ferner `get`, `fail` usw. | Ererbt von istream |

- Operatoren und Manipulatoren — Ererbt von ostream bzw. istream

## 9.5 Statische Elemente

Soll der Wert einer Elementvariablen allen Instanzen einer Klasse gemeinsam sein, so ist diese Elementvariable in ihrer Klasse mit dem Schlüsselwort `static` zu deklarieren.
Sie heißt dann **"statische Elementvariable"** [1].

- Statische Elementvariablen müssen immer zusätzlich wie globale Variablen definiert werden, d.h. außerhalb der Klasse und ohne `static`, aber unter Verwendung des Gültigkeitsbereichs-Operators.
  Dies gilt auch für private Elemente.
- Eine statische Elementvariable wird auch gemeinsam mit allen Instanzen abgeleiteter Klassen verwendet.

Auch eine Funktion kann man mit dem Schlüsselwort `static` deklarieren, wenn deren Wirkung unabhängig vom aktuellen Objekt ist und die daher für alle Instanzen einer Klasse in gleicher Weise arbeitet.
Sie heißt dann **"statische Elementfunktion"**.

- Eine statische Elementfunktion darf vom aktuellen Objekt nur solche Elemente verwenden, die selbst statisch sind.
- In einer statischen Elementfunktion ist kein this-Zeiger definiert.

Der Zugriff auf statische Elementvariablen und -funktionen kann außerhalb der Klasse auch ohne Bezug auf ein Objekt erfolgen. Dann muß aber die Klassenzugehörigkeit mit dem Gültigkeitsbereichsoperator spezifiziert werden.

Ein Zugriff auf statische Elemente ist auf diese Weise sogar möglich wenn noch kein Objekt der Klasse existiert.

Besonderheit: Ganzzahlige Elementvariablen, die mit `static const` definiert sind (und nur solche!), können bereits in der Klassendefinition initialisiert werden.

[1] In Java heißen eine solche Variablen - etwas prägnanter - "Klassenvariablen". Die bisher ausschließlich verwendeten Elementvariablen, bei denen jede Instanz einen selbstständigen Wert besitzt, heißen dort "Instanzvariablen".

## Statische Elemente

**Beispiel zu 9.5**

```
#include <iostream>
using namespace std;

class Artikel {
  public:
    Artikel(int _bestellnr, int _anzahl) { // Konstruktor
      bestellnr = _bestellnr; anzahl = _anzahl;
      gesamtzahl = gesamtzahl + anzahl;     // Gesamtzahl aktualisieren
    }
    void print() {                          // Ausgabefunktion
      cout << "Nr " << bestellnr
           << ": Vorrat " << anzahl
           << ", Gesamtzahl: " << gesamtzahl << endl; }
    int bestellen(int menge);               // Bestellung durchführen
    static int gesamtzahl_print() {         // statische Elementfkt.
      return gesamtzahl; }
  private:
    int bestellnr;                          // Bestellnummer
    int anzahl;                             // vorrätige Stückzahl
    static int gesamtzahl;                  // Gesamtstückzahl im Lager
};

int Artikel::bestellen(int menge) {
  if (anzahl < menge) menge = anzahl;
  anzahl = anzahl - menge;
  gesamtzahl = gesamtzahl - menge;          // Gesamtzahl aktualisieren
  return menge; }

int Artikel::gesamtzahl = 0;                // Definition mit Initial.

// Hauptprogramm
int main() {
  cout << Artikel::gesamtzahl_print();
  Artikel cdrom(1000, 4);
  Artikel monitor(1001, 5);
  cdrom.print();
  monitor.print();
  cout << "Gesamtzahl: " << Artikel::gesamtzahl_print();

}
```

Ausgabe:

# 9.6 Namespaces (Namensbereiche)

Namensbereiche dienen zur Vermeidung von Namenskollisionen in größeren Softwareprojekten.

Deklaration eines Namensbereichs:

```
namespace Namensbereichs-Name {
  Folge von Definitionen und Deklarationen
}
```

Verwenden eines Namens außerhalb seines Namensbereichs:

– Voranstellen des Namensbereichs ("qualifizierter Name")

```
Namensbereichs-Name::Name
```

– Name alleine genügt, wenn dieser zuvor eingeblendet wurde:

a) Einblenden eines einzelnen Namens

```
using Namensbereichs-Name::Name;
```

b) Einblenden aller Namen eines Namensbereichs

```
using namespace Namensbereichs-Name;
```

Globale Namen sind nur noch solche, die auf Dateiebene nicht in einem Namensbereich definiert sind.

Alle Header der Standardbibliothek existieren in der Regel in zwei Versionen:

- Include-Files ohne Endung .h definieren alle Namen im Namensbereich "std". Nur diese sind voll konform zum C++-Sprachstandard.
- Include-Files mit der Endung .h definieren alle Namen im globalen Gültigkeitsbereich. Sie bestehen aus Kompatibilitätsgründen vorerst weiter.

## Artikelklasse in eigenem Namespace

**Beispiel zu 9.6**

```
#include <iostream>
using namespace std;

namespace Lager {
  class Artikel {                                  // Klasse für Artikel
    public:
      void init(int _bestellnr, int _anzahl) {  // Initialisierung
        bestellnr = _bestellnr; anzahl = _anzahl;
      }
      void print() {                               // Ausgabefunktion
        cout << "Nr " << bestellnr
             << ": Vorrat " << anzahl << '\n';
      }
      int bestellen(int menge);                    // Bestellung durchf.
    private:
      int bestellnr;                               // Bestellnummer
      int anzahl;                                  // vorrätige Stückzahl
  };
} // end namespace Lager

int Lager::Artikel::bestellen(int menge) {
  if (anzahl < menge) menge = anzahl;
  anzahl = anzahl - menge;
  return menge;
}

// Hauptprogramm
int main() {
  using Lager::Artikel;
  Artikel cdrom, handbuch;
  cdrom.init(2133, 4);
  handbuch.init(3099, 15);
  cdrom.print();
  int ausgeliefert = cdrom.bestellen(5);
  cout << "Liefermenge: " << ausgeliefert << "\n";
  cdrom.print();
  handbuch.print();
  cin.get();
}
```

Ausgabe:

# 9.7 Klassendeklaration

Wenn sich zwei Klassen A und B wechselseitig aufeinander beziehen, dann muss zur Definition der Elemente von B bereits die Klasse A bekannt sein, aber auch umgekehrt. Um diesen Zirkelschluss zu vermeiden läßt sich mit der "Klassendeklaration" ein Klassenname bekannt machen ohne bereits seine Elemente im einzelnen spezifizieren zu müssen.

- Nach einer Klassendeklaration mit der Syntax

  ```
  class Klassenname;
  ```

  läßt sich die Klasse bereits verwenden – aber nicht in einem Kontext, bei dem Kenntnis der einzelnen Klassenelemente erforderlich ist.

# 9.8 Sichere Casts zwischen Klassen

Wiederholung aus Kap. 5.3:

Eine Typwandlung zwischen zwei Zeigern auf verschiedene Klassen S, R in der Form [1]

**Zeiger auf Typ S** ⇨ **Zeiger auf Typ R**

ist in jedem Fall sicher, wenn R Basisklasse von S ist ("Up-Cast"). Andernfalls kann die Wandlung zu undefinierten Ergebnissen führen, ist damit potentiell gefährlich und erfordert einen expliziten Cast.

Es gibt aber einen Fall, bei dem auch die Typwandlung in umgekehrter Richtung sicher ist, nämlich

- wenn S Basisklasse von R ist ("Down-Cast") und zugleich
- der Zeiger vom Typ S auf eine Instanz der Klasse R (oder einer von R abgeleiteten Klasse) verweist.

Ob diese Situation vorliegt und die Typwandlung eigentlich gefahrlos möglich wäre, läßt sich erst zur Laufzeit entscheiden. Der traditionelle Cast im C-Stil berücksichtigt diese Möglichkeit nicht.
Deshalb besitzt C++ einen weiteren, sicheren Cast, den "dynamischen Cast".

Der Operator

```
dynamic_cast<R*> (Zeiger)
```

wandelt den Zeiger (vom Typ S*) genau dann in einen Zeiger auf R um, wenn diese Umwandlung im obigen Sinne sicher ist.
Ist die Umwandlung nicht sicher, liefert der Operator den Nullzeiger als Ergebnis, was vom Programm getestet werden kann.

- Die Zielklasse muss polymorph sein, d.h. mindestens eine virtuelle Elementfunktion definieren.

[1] S bzw. R stehen für die Klassennamen der Ausgangs-(Source-)Klasse bzw. der Ergebnis-(Result-)Klasse.

## Beispiel zum dynamischen Cast

```
// Klassen gemäß Beispiel "Verwendung einer virtuellen Funktion"
//              (zur Vereinfachung sei Elementvariable kap public)

// Zunächst zwei Objekte und zwei Zeiger definieren,
Artikel cdrom (2133, 4);
Hdisk western (1122, 8, 400);
Artikel* p; Hdisk* hdp;


// 1. Fall: Sicherer Down-Cast
p = &western;          // Up-Cast, immer ok

hdp = (Hdisk*)p;       // Down-Cast (ohne Cast-Ausdruck Compilerfehler)
cout << hdp->kap;      // Ausgabe '400'

// Dasselbe mit dynamischem Cast:
hdp = dynamic_cast<Hdisk*>(p); // hdp jetzt ungleich NULL da sicher
cout << hdp->kap;      // Ausgabe '400'


// 2. Fall: Unsicherer Down-Cast
p = &cdrom;            // Up-Cast, immer ok

hdp = (Hdisk*)p;       // Down-Cast (ohne Cast-Ausdruck Compilerfehler)
cout << hdp->kap;      // Ausgabe zufälliger Wert

// Dasselbe mit dynamischem Cast:
hdp = dynamic_cast<Hdisk*>(p); // hdp jetzt gleich NULL da unsicher
cout << hdp->kap;      // Fehler bei Ausführung, da hdp Nullzeiger
```